Bases de la programmation

Épreuve de TP — Sujet A : gestion des notes d'une promotion

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Épreuve sur machine. Tout votre travail est à rendre dans **un seul fichier** `NOM_Prenom_tpA.py`, déposé à la fin de l'épreuve. Un programme qui ne s'exécute pas (erreur de syntaxe) sera fortement pénalisé : testez chaque fonction au fur et à mesure et commentez ce qui ne fonctionne pas plutôt que de le laisser. Les appels de test sont regroupés dans une fonction `main()`, appelée en fin de fichier par `if __name__ == "__main__":`.

Pour les listes, seules les opérations `len`, `append` et l'accès par indice `[]` sont autorisées (ni `in`, ni `sum`, `max`, `min`, `sort`, `sorted`, tranches…). Accès autorisé à la documentation Python officielle uniquement ; Internet, IA et messageries interdits.

On souhaite écrire un petit logiciel qui gère les notes d'une promotion. Les notes sont rangées dans une matrice `M` : la ligne `i` correspond à l'étudiant numéro `i` et la colonne `j` à la matière numéro `j`. Comme en TD, toute fonction qui reçoit la matrice reçoit aussi `n` (nombre d'étudiants) et `m` (nombre de matières).

Exercice 1 — Saisie et mention *4 points*

1. Écrire une fonction `saisir_note()` qui demande une note réelle à l'utilisateur, la redemande tant qu'elle n'est pas comprise entre 0 et 20 (inclus), puis la renvoie.
2. Écrire une fonction `moyenne(L)` qui renvoie la moyenne des valeurs de la liste `L` (on suppose `L` non vide).
3. Écrire une fonction `mention(moy)` qui renvoie la chaîne correspondant à la moyenne : « Ajourné » (moins de 10), « Passable » (de 10 inclus à 12 exclu), « Assez bien » (de 12 à 14), « Bien » (de 14 à 16) ou « Très bien » (16 et plus).

Exercice 2 — La matrice des notes *5 points*

1. Écrire une fonction `generer_notes(n, m)` qui crée et renvoie une matrice de `n` lignes et `m` colonnes remplie d'entiers tirés au hasard entre 0 et 20 (`from random import randint`).
2. Écrire une fonction `afficher(M, n, m)` qui affiche la matrice avec un numéro d'étudiant en début de ligne :

```
Étudiant 0 : 12 8 15
Étudiant 1 : 5 11 9
Étudiant 2 : 18 14 16
```

3. Écrire une fonction `moyenne_etudiant(M, n, m, i)` qui renvoie la moyenne de l'étudiant `i`, et une fonction `moyenne_matiere(M, n, m, j)` qui renvoie la moyenne de la promotion dans la matière `j`. Vous pouvez réutiliser `moyenne`.
4. Écrire une fonction `bilan(M, n, m)` qui affiche, pour chaque étudiant, sa moyenne (arrondie à 2 décimales avec `round(x, 2)`) et sa mention.

Exercice 3 — Statistiques *4 points*

1. Écrire une fonction `meilleur_etudiant(M, n, m)` qui renvoie deux valeurs : le numéro de l'étudiant ayant la meilleure moyenne (le premier en cas d'égalité) et cette moyenne.
2. Écrire une fonction `nb_admis(M, n, m)` qui renvoie le nombre d'étudiants dont la moyenne est supérieure ou égale à 10.
3. Écrire une fonction `matiere_la_plus_difficile(M, n, m)` qui renvoie le numéro de la matière ayant la plus faible moyenne.

Exercice 4 — Classement *4 points*

On veut afficher le classement des étudiants, de la meilleure moyenne à la moins bonne.

1. Écrire une fonction `liste_moyennes(M, n, m)` qui renvoie la liste des moyennes des `n` étudiants.
2. Écrire une fonction `classement(M, n, m)` qui renvoie la liste des numéros d'étudiants triés par moyenne *décroissante*. On utilisera un *tri à bulles* sur deux listes en parallèle : la liste des moyennes et la liste des numéros `[0, 1, ..., n-1]`. Chaque fois que deux moyennes voisines sont échangées, les deux numéros correspondants sont échangés aussi.
3. Écrire une fonction `afficher_classement(M, n, m)` qui affiche le classement :

```
1er : étudiant 2 (16.0)
2e : étudiant 0 (11.67)
3e : étudiant 1 (8.33)
```

Exercice 5 — Programme principal *3 points*

Écrire la fonction `main()` qui demande le nombre d'étudiants et le nombre de matières (entiers strictement positifs, saisie contrôlée), génère la matrice des notes, puis affiche en boucle le menu suivant jusqu'à ce que l'utilisateur choisisse 0 :

```
1 - Afficher les notes
2 - Bilan par étudiant
3 - Statistiques (meilleur étudiant, admis, matière la plus difficile)
4 - Classement
5 - Modifier une note
0 - Quitter
Votre choix :
```

Un choix invalide affiche « Choix inconnu ». Le choix 5 demande un numéro d'étudiant et un numéro de matière valides, puis la nouvelle note (avec `saisir_note`).

Bonus (+1). Ajouter un choix 6 qui affiche, pour chaque matière, la moyenne, la note minimale et la note maximale de la promotion.

Bases de la programmation

Épreuve de TP — Sujet B : le jeu du morpion

Durée : 1 h 30 • Examen blanc (entraînement)

Consignes. Épreuve sur machine. Tout votre travail est à rendre dans **un seul fichier** `NOM_Prenom_tpB.py`, déposé à la fin de l'épreuve. Un programme qui ne s'exécute pas (erreur de syntaxe) sera fortement pénalisé : testez chaque fonction au fur et à mesure et commentez ce qui ne fonctionne pas plutôt que de le laisser. Le programme est lancé par une fonction `main()`, appelée en fin de fichier par `if __name__ == "__main__":`.

Pour les listes, seules les opérations `len`, `append` et l'accès par indice `[]` sont autorisées (ni `in`, ni `count`, `index`, tranches…). Accès autorisé à la documentation Python officielle uniquement ; Internet, IA et messageries interdits.

Le morpion se joue à deux sur une grille carrée. Chacun à son tour, un joueur place son symbole (`X` pour le joueur 1, `O` pour le joueur 2) dans une case vide. Le premier qui aligne `taille` symboles sur une ligne, une colonne ou une diagonale gagne ; si la grille est pleine sans gagnant, la partie est nulle.

La grille est une matrice carrée `G` de taille `t` × `t` (`t = 3` pour le morpion classique). Chaque case contient `0` (vide), `1` (joueur 1) ou `2` (joueur 2). Toutes les fonctions doivent fonctionner pour n'importe quelle taille `t >= 3`.

Exercice 1 — La grille *4 points*

1. Écrire une fonction `grille_vide(t)` qui crée et renvoie, à l'aide de `append`, une grille `t` × `t` remplie de 0.
2. Écrire une fonction `symbole(v)` qui renvoie `"."` si `v` vaut 0, `"X"` s'il vaut 1 et `"O"` s'il vaut 2.
3. Écrire une fonction `afficher(G, t)` qui affiche la grille avec les numéros de lignes et de colonnes. Exemple pour `t = 3` :

```
  0 1 2
0 X . O
1 . X .
2 . . O
```

Exercice 2 — Jouer un coup *4 points*

1. Écrire une fonction `coup_valide(G, t, i, j)` qui renvoie `True` si la case `(i, j)` existe dans la grille *et* est vide, `False` sinon.
2. Écrire une fonction `saisir_coup(G, t, joueur)` qui affiche le symbole du joueur, demande un numéro de ligne puis un numéro de colonne, et recommence tant que le coup n'est pas valide (en affichant « Coup invalide »). Elle renvoie les deux valeurs `i` et `j`.
3. Écrire une fonction `grille_pleine(G, t)` qui renvoie `True` si aucune case n'est vide.

Exercice 3 — Détection du gagnant *5 points*

1. Écrire une fonction `ligne_gagnante(G, t, i, joueur)` qui renvoie `True` si toutes les cases de la ligne `i` appartiennent à `joueur`.
2. Écrire de même `colonne_gagnante(G, t, j, joueur)`.
3. Écrire une fonction `diagonale_gagnante(G, t, joueur)` qui teste les *deux* diagonales (la diagonale principale contient les cases `(k, k)`, l'autre les cases `(k, t-1-k)`).
4. Écrire une fonction `a_gagne(G, t, joueur)` qui renvoie `True` si `joueur` a gagné, en utilisant les trois fonctions précédentes.

Exercice 4 — Une partie *4 points*

1. Écrire une fonction `partie(t)` qui joue une partie complète entre deux joueurs humains : elle crée la grille, puis, tant que la partie n'est pas terminée, affiche la grille, fait jouer le joueur courant et change de joueur. À la fin, elle affiche la grille et le résultat (« Le joueur X a gagné » ou « Match nul »), et renvoie le numéro du gagnant (1 ou 2), ou 0 en cas de match nul.
2. Écrire la fonction `main()` qui demande la taille de la grille (entier entre 3 et 6, saisie contrôlée), enchaîne les parties tant que l'utilisateur répond exactement `"o"` à la question « Une autre partie (o/n) ? », et tient le score. Avant de s'arrêter, elle affiche le nombre de victoires de chaque joueur et le nombre de matchs nuls.

Exercice 5 — Contre l'ordinateur *3 points*

On veut maintenant que le joueur 2 soit l'ordinateur.

1. Écrire une fonction `coup_aleatoire(G, t)` qui renvoie une case vide tirée au hasard (on tire `i` et `j` avec `randint` jusqu'à obtenir une case vide ; on suppose la grille non pleine).
2. Écrire une fonction `coup_ordinateur(G, t)` un peu plus maligne : si l'ordinateur peut gagner en un coup, il joue ce coup ; sinon, si le joueur 1 peut gagner au coup suivant, il bloque cette case ; sinon, il joue au hasard.
   Indication : pour tester une case vide, on y place provisoirement le joueur, on appelle `a_gagne`, puis on remet la case à 0.
3. Modifier `partie` (ou en écrire une copie `partie_ordi`) pour que le joueur 2 utilise `coup_ordinateur`, et proposer ce mode de jeu dans `main`.

Sujet 0 — Examen de TP

Bases de la programmation – Révision des TP 1, 2 et 3

Durée : 2 h — Barème indicatif : 20 points

Consignes.

- Créez un répertoire `EXAM_TP` et écrivez tout votre travail dans **un seul fichier** `NOM_Prenom.py`. Exécutez-le avec `python3 NOM_Prenom.py`.
- Respectez **exactement** les noms de fonctions et l'ordre des paramètres demandés.
- Une fonction qui « renvoie » utilise `return` et **n'affiche rien** ; seules les fonctions explicitement « d'affichage » ou « de saisie » utilisent `print` / `input`.
- Tout le code de test est placé dans une fonction `main()` appelée à la fin du fichier par `if __name__ == "__main__": main()`.
- Comme en TP 3, sur les listes seules les opérations `len`, `append`, `insert` et l'accès par indice `[]` sont autorisées (ni `in`, ni `remove`, `count`, `index`, tranches…). Les fonctions `min`, `max`, `sum`, `sort`, `sorted`, `abs` sont interdites. Seul `split` est autorisé, uniquement pour la saisie de l'exercice 13.
- Un fichier qui ne s'exécute pas (erreur de syntaxe) n'est pas corrigé : commentez ce qui ne marche pas.

Partie A — Conditions et saisies (TP 1) — 5 points

Dans cette partie, les boucles sont interdites. Chaque exercice est écrit dans une fonction d'affichage sans paramètre (`exo1()`, `exo2()`, …).

Exercice 1 Le plus grand et le plus petit (1,5 pt)

Écrire `exo1()` qui lit trois entiers `a`, `b` et `c`, puis affiche le plus grand et le plus petit des trois. Si les trois valeurs sont égales, elle affiche seulement « Les trois valeurs sont égales ».

Exercice 2 Calculatrice (1,5 pt)

Écrire `exo2()` qui demande une opération parmi `+`, `-`, `*`, `/` et `%`, puis deux entiers, et affiche le résultat (`/` donne le quotient entier `//`). Pour `/` et `%`, la division par zéro doit être interdite (« Division par zéro impossible »). Si l'opération est inconnue, afficher « Opération inconnue ».

```
Entrez l'opération : %
Entrez le premier opérande : 17
Entrez le deuxième opérande : 5
Le résultat de 17 % 5 est 2
```

Exercice 3 Nombre à trois chiffres (1 pt)

Écrire `exo3()` qui lit un entier. S'il ne contient pas exactement trois chiffres, afficher « Le nombre ne contient pas 3 chiffres ». Sinon, afficher ses trois chiffres séparés par des espaces, puis « palindrome » s'il se lit de la même façon dans les deux sens (par exemple `121`), « pas palindrome » sinon.
Exemple : pour `472`, afficher `4 7 2` puis `pas palindrome`.

Exercice 4 Nature d'un triangle (1 pt)

Écrire `exo4()` qui lit les coordonnées entières de trois points du plan. Elle affiche « Deux points sont confondus » si c'est le cas. Sinon, elle affiche « Triangle rectangle » et/ou « Triangle isocèle » selon le cas (un triangle peut être les deux), ou « Triangle quelconque ». On travaillera uniquement avec les **carrés** des longueurs (aucune racine carrée).

Partie B — Les fonctions (TP 2) 8 points

Exercice 5 Années bissextiles (1,5 pt)

A. Écrire `est_bissextile(annee)` qui renvoie `True` si l'année est bissextile : divisible par 4 mais pas par 100, ou divisible par 400.
B. Écrire `nb_jours(mois, annee)` qui renvoie le nombre de jours du mois (numéroté de 1 à 12) pour l'année donnée, en utilisant `est_bissextile`.
Exemples : `nb_jours(2, 2024)` renvoie 29, `nb_jours(2, 1900)` renvoie 28, `nb_jours(11, 2025)` renvoie 30.

Exercice 6 Équation du second degré (2 pt)

A. Écrire `resoudre(a, b, c)` qui résout $ax^2 + bx + c = 0$ pour **tous** réels a, b, c (y compris $a = 0$ ou $b = 0$), **sans aucun affichage**. Elle renvoie trois valeurs `nb, x1, x2` : `nb` vaut -1 si tous les réels sont solutions, sinon 0, 1 ou 2 ; `x1 < x2` s'il y a deux solutions ; une valeur inutilisée vaut `None`.
Exemples : `resoudre(1, -3, 2)` renvoie `(2, 1.0, 2.0)` ; `resoudre(0, 2, 1)` renvoie `(1, -0.5, None)` ; `resoudre(0, 0, 0)` renvoie `(-1, None, None)`.
B. Écrire la fonction d'affichage `afficher_solutions()` qui demande a, b, c, appelle `resoudre` et affiche une phrase adaptée à chacun des cas.

Exercice 7 Nombres premiers et décomposition (2 pt)

A. Écrire `est_premier(n)` qui renvoie `True` si n est premier.
B. Écrire `plus_petit_diviseur(n)` qui renvoie le plus petit diviseur premier de $n > 1$, en utilisant `est_premier`.
C. Écrire `decomposition_puissances(n)` qui renvoie la décomposition de $n > 1$ en facteurs premiers sous forme de chaîne, **en regroupant les facteurs égaux avec un exposant** (exposant écrit seulement s'il est > 1).
Exemples :

- `decomposition_puissances(360)` renvoie `"2^3 x 3^2 x 5"` ;
- `decomposition_puissances(12)` renvoie `"2^2 x 3"` ;
- `decomposition_puissances(97)` renvoie `"97"`.

Exercice 8 Factorielle et triangle de Pascal (1,5 pt)

A. Écrire `factorielle(n)` (avec $0! = 1$) et `coeff_binomial(n, k)` qui renvoie $\binom{n}{k} = \dfrac{n!}{(n-k)!\,k!}$ sous forme d'**entier**.
B. Écrire `ligne_pascal(n)` qui **renvoie** la ligne numéro n du triangle de Pascal sous forme de chaîne, les nombres étant séparés par un espace. *Exemple :* `ligne_pascal(4)` renvoie `"1 4 6 4 1"`.
C. Écrire `triangle_pascal(nb_lignes)` qui **affiche** les `nb_lignes` premières lignes, en utilisant `ligne_pascal`.

Exercice 9 Placement à intérêts (1 pt)

Un livret rapporte chaque mois un pourcentage `taux` de sa valeur (par exemple `0.02` pour 2 %). Écrire `mois_pour_atteindre(depart, taux, cible)` qui renvoie le nombre de mois nécessaires pour que la valeur atteigne ou dépasse `cible`.
Exemple : `mois_pour_atteindre(100, 0.02, 150)` renvoie 21.

Partie C — Les listes : un mini-Mastermind (TP 3) 7 points

Exercice 10 Outils sur les listes (2 pt)

A. Écrire `contient(L, v)` qui renvoie `True` si `v` figure dans `L` (l'opérateur `in` est interdit).
B. Écrire `sans_element(L, i)` qui renvoie une **nouvelle** liste égale à `L` privée de l'élément d'indice `i`. *Exemple :* `sans_element([7, 3, 9, 1], 1)` renvoie `[7, 9, 1]`.
C. Écrire `chiffres_differents(L)` qui renvoie `True` si tous les éléments de `L` sont différents.

Exercice 11 Code secret (1 pt)

Écrire `init(n, k)` qui renvoie une liste de `n` chiffres **tous différents** tirés au hasard dans $[1, k]$ (on suppose $k \geqslant n$), en utilisant `contient`.

Exercice 12 Score d'une proposition (1,5 pt)

Écrire `score(H, R)` qui renvoie deux valeurs : le nombre de chiffres bien placés (`H[i] == R[i]`) et le nombre de chiffres présents dans `R` mais mal placés.
Exemples avec `R = [1, 2, 4, 8, 5]` : `score([1, 2, 3, 4, 5], R)` renvoie `(3, 1)` ; `score([1, 2, 3, 6, 4], R)` renvoie `(2, 1)`.

Exercice 13 Saisie et partie (2,5 pt)

A. Écrire `saisir_proposition(n)` qui lit sur une seule ligne des chiffres séparés par des espaces (avec `split`), **redemande** tant qu'il n'y a pas exactement `n` valeurs, puis renvoie la liste d'entiers correspondante.
B. Écrire `jeu(n, k, max_essais)` qui tire un code avec `init`, puis fait proposer l'utilisateur en affichant le score à chaque essai. La partie s'arrête sur « BRAVO !! » (avec le nombre d'essais) ou, après `max_essais` échecs, sur « Perdu, le code était » suivi du code.
C. Compléter `main()` avec un menu qui boucle tant que l'utilisateur ne choisit pas `0` :

```
1 : Mastermind (n = 4, k = 6, 10 essais)
2 : Triangle de Pascal
3 : Résoudre une équation
0 : Quitter
Votre choix : 1
Entrez une liste de chiffres : 1 2 3
Il faut exactement 4 valeurs.
Entrez une liste de chiffres : 1 2 3 4
2 bien placés, 1 mal placé
...
```

Bonus — L'ordinateur devine (TP 3, exercice 2) +2 points

On dispose de la fonction `toutes_possibilites(n, longueur)` du TP 3, à recopier telle quelle.

A. Écrire `candidats(n, longueur)` qui renvoie uniquement les possibilités formées de chiffres différents (utiliser `chiffres_differents`). Vérifier que `len(candidats(4, 3))` vaut `24`.
B. Écrire `devine(longueur, n)` qui propose à chaque essai le premier candidat restant, demande à l'utilisateur le nombre de bien et de mal placés, ne garde que les candidats compatibles avec **toutes** les réponses déjà données, et renvoie le code trouvé et le nombre d'essais.